



Open Weights, Bought Data, Rented Gradients: Training as a Market Workload

Zach Kelling

Hanzo AI

2026

Abstract

A compute market that sells only inference is selling the cheap half. Training is where the capital goes, where the differentiation lives, and where a decentralized provider network has a structural advantage that inference does not give it — training tolerates latency, tolerates heterogeneity, and is checkpointable, which means it can use exactly the supply that interactive serving cannot.

It also has a verification problem that is worse than inference’s and a data problem that inference does not have at all. This paper treats all three. We describe why open-weight models are the only viable substrate for a decentralized training market, what the training workload actually looks like when decomposed into market-tradable units, how gradient work can be checked without recomputing it, and how data becomes a purchasable input with provenance that survives into the resulting weights.

The enabling development is that confidential computing on current accelerators has become close to free for the workloads that matter. That changes the design: a provider can be given proprietary weights and licensed data, prove it ran the declared computation inside an attested boundary, and be unable to exfiltrate either. Training on rented hardware stops requiring that the renter be trusted.

Contents

1	Why open weights are the substrate	3
2	Decomposing the workload	3
3	Checking gradient work	4
4	Confidential computing changes the design	4
4.1	What this makes possible	5
4.2	What it does not do	5
5	Data as a purchasable input	5
6	What exists	6
7	Sequencing	6

1 Why open weights are the substrate

A decentralized training market cannot be built on closed models, and the reason is structural rather than ideological. A closed model reachable only through a vendor’s API cannot be fine-tuned on a third party’s accelerator, because the weights never leave the vendor. Whatever a market does with such a model, it is reselling an API, and the provider network is decoration.

Open-weight models invert this. The weights are an artifact anyone can hold, so any provider with sufficient memory is a candidate supplier, and the market has something real to allocate. Everything in this paper assumes open weights for that reason.

This produces a natural division of labour with a routing layer over closed frontier models. The closed family is where the best answer often comes from today, and routing across it is a real product. The open family is where a decentralized network can actually do work — serve on arbitrary hardware, be fine-tuned per tenant, be distilled, be quantized, be improved by anyone. A market that carries both routes intelligence at the top and allocates genuine compute underneath.

2 Decomposing the workload

Training is not one job. It decomposes into units with sharply different market properties, and a market that treats them alike will misprice all of them.

Unit	What it is	Latency	Parallel	Verifiable
Adapter fit	low-rank delta on frozen base	tolerant	across tenants	partly
Preference optimisation	direct preference and its relatives	tolerant	across pairs	partly
Full fine-tune	all weights, sharded	tolerant	across shards	hard
Distillation	student fits teacher outputs	tolerant	across batches	yes
Synthetic generation	teacher produces training data	tolerant	embarrassingly	yes
Evaluation	scored inference on a fixed set	tolerant	embarrassingly	yes
Quantization	post-training weight compression	tolerant	across tensors	yes

Table 1: Training units and their market properties.

Three of these are worth singling out because they are the easy money and are routinely overlooked.

Synthetic generation is inference. Producing training data from a teacher model is a batch inference workload with no latency constraint and no interactivity. It is the single best fit for cheap heterogeneous supply in the entire stack, and it verifies under exactly the determinism discipline the companion paper describes — pin the seed, pin the numerics, and the output is reproducible.

Evaluation is inference on a fixed set. Scoring a candidate model against a benchmark is deterministic given a pinned harness. A market can sell evaluation as a service with byte-equality verification and no trust requirements, which also gives it something valuable: an independent measurement of every model in its own catalogue.

Adapter fitting parallelises across tenants, not within. Ten thousand tenants each fitting a small adapter is ten thousand independent jobs. That is a better fit for a heterogeneous network than one large job needing tight interconnect, and it is the workload personalization actually generates.

By contrast, a full fine-tune of a large model needs high-bandwidth interconnect between shards, which is precisely what a geographically dispersed network does not have. Honest market design puts full pretraining and large full fine-tunes in the datacentre and takes the rest.

3 Checking gradient work

Training’s verification problem is harder than inference’s. An inference result can at least be spot-checked algebraically against the weights that produced it. A gradient step’s output is a weight update, and a provider that returns a plausible but wrong update degrades the model subtly over many steps rather than failing visibly once.

Four mechanisms are available, and they stack.

Loss trajectory as a cheap signal. The buyer holds out a small validation set and evaluates the returned checkpoint. A provider submitting garbage is caught immediately; a provider submitting a slightly-wrong update is not. This is a liveness check, not a correctness check, and should be described as such.

Recomputation of sampled steps. The buyer recomputes a random fraction of steps from the checkpoint and the batch. This requires the step to be deterministic — pinned numerics, pinned data order, pinned dropout seed — which is the same discipline as everywhere else in this series. Cost is the sampling rate; soundness is the sampling rate. With stake at risk, a low rate is adequate, and it is the workhorse mechanism.

Algebraic checks on the dominant operation. The backward pass is dominated by matrix products, and those are checkable in quadratic rather than cubic time by randomised verification, at well under a percent of the cost of the product for the dimensions in question. This checks the expensive part and is silent about the rest.

Attested execution. Discussed next, and increasingly the answer.

4 Confidential computing changes the design

The development that reorganises this problem is that hardware confidential computing on current accelerators has become cheap enough to use by default.

For compute-bound large-model work — which is what training and large-batch inference are — the overhead of running inside an encrypted memory boundary is small, because the cost is dominated by arithmetic rather than by transfers across the boundary. Current-generation hardware with encrypted device links narrows the remaining gap further by removing the staging copies that earlier designs required. Small, transfer-bound workloads still pay meaningfully; the workloads a compute market sells largely do not.

We classify providers by what their hardware can actually attest, in four tiers, with all attestation performed locally rather than against a vendor cloud service — a market that cannot verify without an external dependency has not removed the trusted party, it has renamed it.

Tier	Hardware	What it attests	Trust band
1	accelerator-native confidential mode	device, firmware, encrypted memory	90–100
2	confidential virtual machine plus accelerator	host measurement plus device	70–89
3	device secure enclave with an inference engine	enclave measurement	50–69
4	no hardware support	nothing; stake substitutes	10–49

Table 2: Provider tiers by attestable capability. An attestation carries the device identity, model, whether confidential mode and encrypted device links are enabled, the partitioning configuration, and driver and firmware versions.

4.1 What this makes possible

Training on weights the provider may not keep. A model owner ships proprietary weights into an attested boundary. The provider supplies accelerators and receives payment; it does not receive the weights in any form it can extract. This is the difference between a market that can only train public models and one that can train anything.

Training on licensed data. The same argument applies to the data, and it matters more, because data licences are typically far more restrictive than model licences. A corpus that may not be copied can still be trained on inside a boundary that can prove it did not leave.

Verification for nondeterministic work. This is the deepest consequence. Byte-equality verification covers bit-reproducible work and stops there. Attested execution covers the rest: it proves the declared computation ran on declared hardware with declared inputs, which is exactly the class of claim that no algebraic check can make. At a small percentage overhead this is dramatically cheaper than replication, and it applies where the cheap checks cannot reach.

4.2 What it does not do

Attestation moves the trust root to a silicon vendor’s certificate chain. That is a real and different assumption, not an elimination of assumptions, and the history of side-channel and fault attacks against such boundaries argues against using attestation as the sole mechanism for the highest-value work. Used together with sampled recomputation and algebraic checks it is excellent; used alone at tier 1 it is a strong claim resting on one vendor’s supply chain.

5 Data as a purchasable input

Inference buys compute. Training buys compute and data, and the data side is the one most markets have no mechanism for at all.

The paradox. Data’s value cannot be assessed without inspecting it, and inspecting it is most of what a buyer wanted to pay for. The mechanisms that resolve this are all forms of buying a *function* of the data rather than the data itself.

Train-without-transfer. The buyer sends the model into the attested boundary; the seller supplies data that never leaves it; the buyer receives the updated weights. The seller is paid for the improvement rather than for the corpus, and retains it. This is the mechanism confidential computing unlocks, and it is the reason the two halves of this paper belong together.

Evaluation before purchase. The seller permits a scored evaluation run against a small sample, producing a measured improvement figure that the buyer can use to price the whole. This requires trusting that the sample represents the corpus, which is a bounded and familiar commercial risk.

Provenance that survives into the weights. A training run should record which corpora contributed, under what licence, in what proportion, with content-addressed identifiers — carried with the resulting checkpoint. This is becoming a compliance requirement rather than a nicety, and a market that records it structurally is worth more to any buyer who will ever be asked where their model came from. It also creates the ledger against which contributors can be paid when a model is later commercialised, which is the only credible basis for recurring data compensation.

Contribution measurement. Attributing model quality to individual data sources is genuinely unsolved at scale — influence functions and leave-one-out retraining are both far too expensive for large corpora. We do not claim otherwise. Proportional payment by tokens contributed, weighted by a measured improvement on held-out evaluation for the corpus as a whole, is crude, cheap and implementable, and it is better than the alternative of paying nothing.

6 What exists

The training side of our stack is open source and in production use.

A fine-tuning framework carrying low-rank and quantized low-rank adaptation, direct preference optimisation and its group-relative, odds-ratio and Kahneman–Tversky variants, proximal policy optimisation, sharded data-parallel and pipeline execution, sample packing and fused attention — the standard corpus of open-weight training methods, maintained as a non-profit research project.

A training crate inside the inference engine implementing trainable low-rank layers over frozen bases, an optimiser, distillation and adapter serialisation whose tensor layout matches the inference-side merge, exposed over the serving API as a session-oriented loop: create a client against a base model, accumulate gradients, step, sample from the current weights, save the adapter.

Provider classification and local attestation verification for the four tiers above, including accelerator attestation parsing.

And the pricing and routing layer that quotes all of it, described in the companion paper on the market mechanism.

7 Sequencing

Synthetic generation and evaluation first: both are inference, both verify under byte equality, both tolerate arbitrary latency, and both have immediate demand from anyone training anything. Adapter fitting next, because personalization generates it in volume and it parallelises across tenants rather than within a job. Distillation and quantization after that. Attested training on proprietary weights and licensed data once tier-1 supply is a meaningful share of the network — that is the highest-value workload here, and it is the one that requires the hardware story to be real rather than aspirational.

Large full fine-tunes and pretraining stay in the datacentre. Saying so plainly is worth more than a claim that will be tested and fail.